

个性化健康规划多智能体平台

—— 基于 LangGraph 的 Supervisor-Worker 多智能体协作系统

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	个性化健康规划多智能体平台
方向	Agentic AI 原生开发
学号	2025303062
姓名	李晨坤
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

- [一、选题背景与设计思想](#)
- [二、Specs 规格文档](#)
- [三、系统架构与设计](#)
- [四、关键实现与代码展示](#)
- [五、测试与评估](#)
- [六、系统升级与扩展](#)
- [七、课程总结](#)

一、选题背景与设计思想

1.1 问题定义

健康管理是一个典型的多维度交叉问题。饮食、运动、睡眠三者之间存在深刻的生理学关联：睡眠不足会导致饥饿素上升约 28%、瘦素下降约 18%，增加高热量食物摄入；规律运动则能增加深度睡眠时长，形成正向循环。然而，现有的健康管理工具将这三者割裂处理——用户需要在薄荷健康记录饮食、在 Keep 完成训练、在 Sleep Cycle 查看睡眠，没人帮他们整合。

更深层的问题是，现有工具普遍采用规则引擎驱动。它们能告诉你"BMI 24 属于偏胖"，但无法理解"我最近加班压力大，晚上总暴食，体重涨了 5 斤，怎么调整？"这种带有因果链和场景的复杂表述。这正是大语言模型和 Agent 技术的用武之地——不是替代规则引擎，而是让 AI 像真正的健康顾问一样，先理解你的处境，再给出针对性的建议。

1.2 现有方案的不足

方案类型	代表	核心缺陷
通用健康 App	Keep、薄荷健康	规则驱动，无法理解复杂语义；饮食运动数据割裂
通用 AI 聊天	ChatGPT	缺乏领域工具（无法计算 BMR/TDEE）；无多 Agent 协作
单一 Agent	Health Bot	只覆盖一个领域；无跨领域协同机制
企业健康平台	体检中心系统	人工依赖重；缺乏持续追踪和动态调整

核心空白在于：缺少一个能**同时理解饮食、运动、睡眠**三个领域、**记住用户偏好和进展**、**调用专业工具**进行计算分析、**多专家协作**输出综合方案的 AI 系统。

1.3 项目价值与定位

本项目的定位不是做"更好的饮食 App"或"更智能的健身教练"，而是构建一个**多专家 AI 协作系统**，模拟真实世界的多学科会诊。在医疗场景中，全科医生先了解患者整体情况，再判断需要哪些专科医生（营养师、体能教练、睡眠医生），各专家独立诊断后，全科医生整合各方意见、检查矛盾、形成统一方案。

技术层面，Consultation Agent 担任这个"全科医生"角色，Diet/Exercise/Sleep Agent 分别担任三个"专科医生"。系统还实现了 Reflection 质量门禁——在所有专家给出建议后自动检查置信度、一致性和完整性，确保最终输出是一个内部协调的整体方案而非简单拼接。

1.4 技术路线

选择**方向一：Agentic AI 原生开发**。核心技术要素按课程要求 6/6 全覆盖，并额外集成 MCP 协议和 Agentic RAG 两项加分技术。

要素	实现方式
SDD 规格驱动	Pydantic v2 定义 30+ 数据模型，模型即可执行规格
工具使用 / MCP	19 个 Function Calling 函数 + 3 个 MCP 健康知识工具
记忆机制	三层架构：短期缓冲 + ChromaDB 向量语义记忆 + JSON 持久化
状态管理与推理	LangGraph StateGraph + Supervisor-Worker + ReAct + Reflection
多智能体协作	4 Agent（Consultation / Diet / Exercise / Sleep）并行协作
可观测性与评估	Langfuse 追踪 + Token/成本统计 + 10 项评估基准
+ MCP 协议	健康指南查询 / 药物相互作用检查 / 医学参考范围
+ Agentic RAG	20+ 篇循证文档，覆盖 6 大健康领域

二、Specs 规格文档

SDD 方法论的核心是“规格即代码”——Pydantic 模型既是设计文档，又是运行时校验层。本章展示三个核心规格。

2.1 Product Spec

核心实体 `UserProfile` 定义了四个信息维度：基本身份（年龄、性别）、当前身体测量（身高、体重、可选体脂率和腰围）、生活偏好与约束（活动水平五级、饮食偏好、过敏、运动偏好、可用器材）、历史与元数据（测量历史列表、时间戳）。

一个重要的设计决策是将身体测量封装为独立的 `BodyMetrics` 模型而非平铺字段。原因是用户的体重随时间变化，每次记录体重需要附带测量日期。`UserProfile` 中的 `metrics_history: list[BodyMetrics]` 字段自然支持了体重趋势追踪功能，不需要额外的数据库设计。

功能需求方面，系统定义了 10 项功能，按 P0-P2 分级。P0 是核心闭环：健康档案 + 营养评估 + 运动处方 + 睡眠分析 + 多领域协同。P1 是增强功能：药物安全检查、循证知识检索、进展追踪。P2 是体验优化：长期记忆、流式响应。非功能需求包括 API 调用失败自动重试（指数退避）、连续 5 次失败熔断 30 秒、代码中零硬编码 API Key 等。

2.2 Architecture Spec

三个 Pydantic 模型定义了 Agent 间的通信协议：

`AgentMessage` 是专业 Agent 向 Supervisor 发送的消息格式。除了回复文本外，还包含置信度评分（0-1）、使用的工具列表、信息来源和注意事项。其中置信度是 Reflection 质量门禁的关键输入——当某个 Agent 的置信度低于 0.4 时，系统会在最终回复中标注不确定性。

`SupervisorDecision` 定义路由决策。`requires_reflection` 标志在当前策略下当涉及 2 个及以上 Agent 时自动为 True——多 Agent 协同产出存在内部矛盾的风险更高。`priority` 字段区分常规咨询和紧急情况（如用户描述胸痛时直接建议就医，不调用任何 Agent）。

`AgentResponse` 是最终返回给用户的回复体。包含健康警告列表（BMI 超阈值自动弹窗）、综合置信度（HIGH/MEDIUM/LOW 三级，阈值 0.85/0.6）和免责声明。

2.3 API Spec

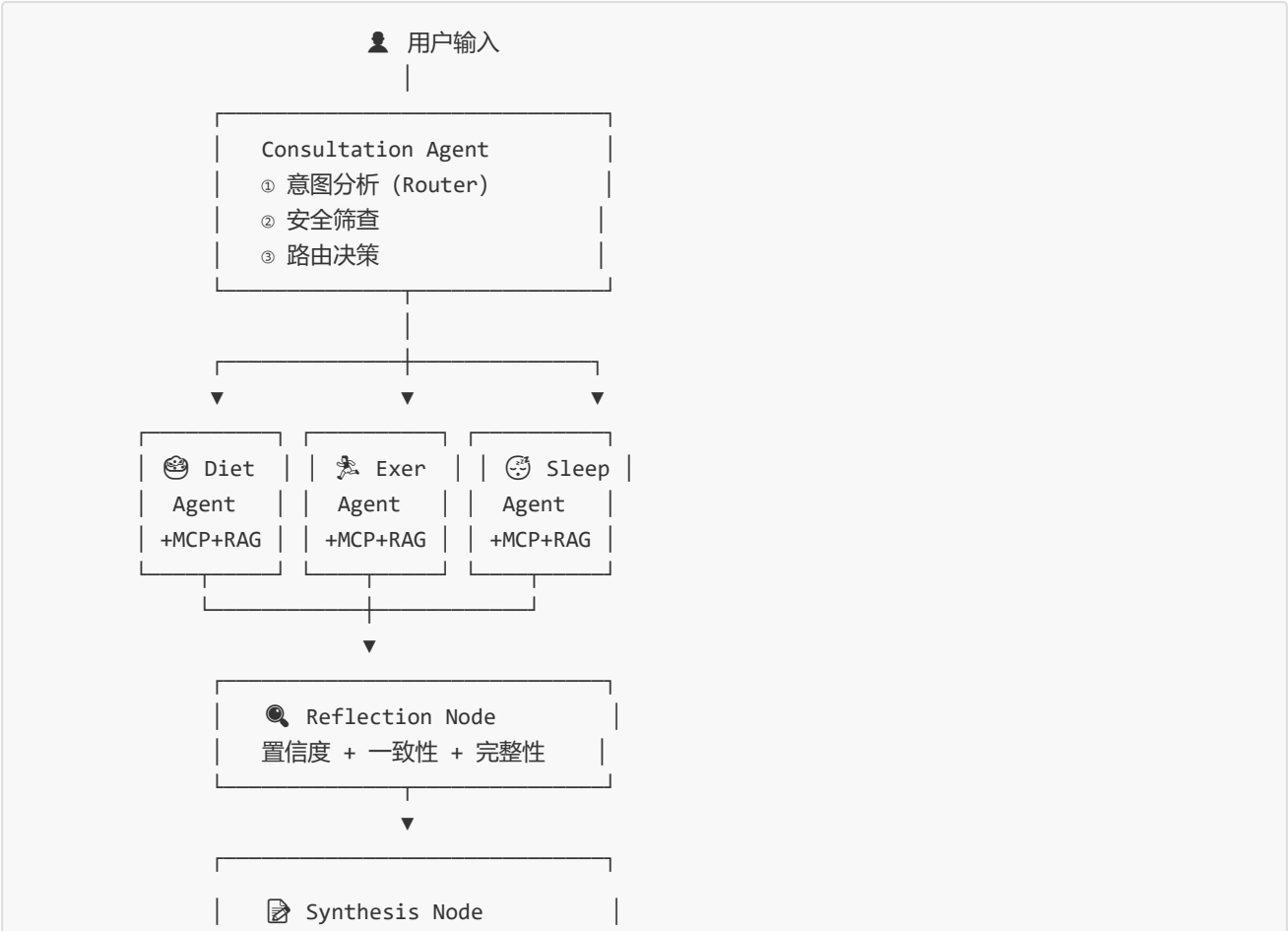
LangGraph 状态定义 `HealthGraphState` 采用分层设计：对话层（messages、user_input）、用户上下文层（user_profile、health_goals）、编排控制层（supervisor_decision、agent_outputs）、质量控制层（reflection_needed、quality_gate_passed）和元数据层（trace_id、token_usage、errors）。各层字段互不干扰，每个图节点只读写自己负责的字段。

对外服务通过 FastAPI 提供 6 个 RESTful 端点（`/chat`、`/chat/stream`、`/profile`、`/health`、`/stats`、`/eval`），所有请求/响应模型均为 Pydantic 类，访问 `/docs` 即可获得交互式 OpenAPI 文档。

三、系统架构与设计

3.1 核心架构

系统采用 Supervisor-Worker + Reflection 架构，灵感来源于真实世界多学科会诊的工作模式。



3.2 完整交互流程

以用户提问“我想科学减重 10 公斤”为例，展示从输入到输出的完整链路。

阶段一：意图分析与路由。 Consultation Agent 委托 IntentRouter 分析用户输入。Router 采用双层策略：先做关键词匹配——遍历三组关键词表（Diet 50+、Exercise 50+、Sleep 50+ 个中英文关键词），计算各 Agent 得分；再做协同触发——“减重”一词命中协同规则，自动将 Exercise 加入路由（因为减重的本质是热量缺口，需要饮食和运动协同）。安全筛查优先级最高：一旦命中“胸痛”“呼吸困难”“自杀”等危急关键词，direct 返回就医建议，跳过所有后续流程。

阶段二：并行 Agent 执行。 Diet Agent 收到请求后进入 ReAct 循环。它先推理“我需要用户的基础代谢率”，调用 `calculate_bmr` 获取结果（如 1648.75 kcal），再推理“还需要每日总消耗”，调用 `calculate_tdee`。接着可能调用 `search_health_knowledge` 检索循证减重指南，最后调用 `generate_meal_plan` 生成一日三餐计划。Exercise Agent 同步进行类似流程。两个 Agent 并行执行，总延迟从串行的 6-10 秒缩短到 3-5 秒。

阶段三：质量审查。 Reflection 节点检查三个维度：置信度是否达标（低于 0.4 标注不确定性）、是否存在矛盾（如“低碳水”和“高碳水”同时出现）、是否所有预期 Agent 都产出了结果。通过后进入 Synthesis。

阶段四：结果合成。 Synthesis 节点将 Diet 和 Exercise 的输出融合为一份连贯回复：先概述用户体重现状（BMI 24.2，偏胖），给出热量目标（日摄入 1500 kcal，缺口 500 kcal），展示饮食+运动协同方案（饮食减少 500 kcal 摄入 + 运动消耗增加 300 kcal），最后附上 3-5 条本周可执行的具体行动。

3.3 关键设计决策

为什么提示词放在 YAML 文件中？ Agent 的行为由提示词决定。将提示词提取为独立的 YAML 配置文件，意味着产品人员可以直接调整 Agent 的角色定位或回复风格而不需要改代码。PromptManager 内置缓存和 `clear_cache()` 热更新接口。四个 Agent 的 YAML 文件各自独立，修改 Diet Agent 不影响其他 Agent。

为什么需要三层记忆架构？ 不同的记忆需求有不同的性能特性。短期缓冲（当前会话）需要毫秒级读取，提供对话连贯性。向量语义记忆（ChromaDB）支持跨会话检索——用户一周后回来问“上次的饮食计划怎么调整”时，系统能从向量库中检索到上次对话。JSON 持久化保证档案、目标等结构化数据不丢失。三层各司其职，避免“一刀切”方案带来的性能和功能妥协。

为什么选择 Reflection 而非让 LLM 直接输出？ 让一个 LLM 同时扮演多个专家会导致“知识混淆”。每个 Agent 用各自的 system prompt 约束知识范围，独立分析后再由 Reflection 做质量校验，本质上是“分离关注点 + 事后校验”，比“事前约束一个 LLM 什么都懂”更可靠。

四、关键实现与代码展示

4.1 Agent 核心循环（ReAct）

以下是 BaseAgent 的核心调用逻辑，每个 Agent 通过继承获得完整的 LLM + 工具调用能力：

```

def invoke(self, user_message, context) -> AgentMessage:
    if self.circuit.is_open:                # ① 熔断检查
        return self._fallback("熔断器开启")

    system_prompt = self._build_system_prompt(context) # ② 加载 YAML 提示词
    messages = [{"role":"system","content":system_prompt},
                {"role":"user","content":user_message}]

    for _ in range(MAX_TOOL_ROUNDS):        # ③ ReAct 循环 (≤5轮)
        msg = self._call_llm(messages)      # 带指数退避重试
        if not msg.tool_calls: break        # LLM 决定不再需要工具
        messages = self._execute_tools(messages, msg) # ④ 执行工具调用

    return AgentMessage(                   # ⑤ 结构化输出
        content=msg.content,
        confidence=extract_confidence(msg.content), # 自动解析置信度
        sources=extract_sources(msg.content))      # 自动提取引用来源

```

循环上限设为 5 轮是一个工程权衡：太少了复杂问题分析不充分，太多了 LLM 可能在工具调用中"迷失"并且增加费用。实验表明大多数健康咨询在 1-3 轮完成。

4.2 工具系统

19 个工具函数分为 6 组，每组有独立的工具 Schema 和实现。以 Diet Agent 的 BMR 计算为例，工具实现是纯函数（无副作用、可独立测试），Schema 遵循 OpenAI Function Calling 标准：

```

# 工具实现
def calculate_bmr(weight_kg, height_cm, age, gender) -> dict:
    if gender == "male":
        bmr = 10 * weight_kg + 6.25 * height_cm - 5 * age + 5
    else:
        bmr = 10 * weight_kg + 6.25 * height_cm - 5 * age - 161
    return {"bmr": round(bmr, 1), "formula": "Mifflin-St Jeor"}

# Function Calling Schema
{"type":"function","function":{"
    "name":"calculate_bmr",
    "description":"使用 Mifflin-St Jeor 公式计算基础代谢率",
    "parameters":{"type":"object","properties":{"
        "weight_kg":{"type":"number"},"height_cm":{"type":"number"},
        "age":{"type":"integer"},"gender":{"enum":["male","female","other"]}
    },"required":["weight_kg","height_cm","age","gender"]}}
}}

```

MCP 工具组（`query_health_guidelines`、`check_drug_interactions`、`get_medical_reference`）和 RAG 工具（`search_health_knowledge`）使 Agent 的回复从"凭经验"升级为"引证权威来源"，知识库中 20+ 文档全部标注了来源和可靠性评分。

4.3 提示词与代码分离

四个 YAML 文件各自定义 Agent 的身份、专业领域、工作流程和回复规范。`PromptManager` 负责加载、缓存和格式化。修改 Agent 行为只需编辑 YAML：

```
# prompts/diet_agent.yaml (节选)
role: "饮食健康专家"
expertise:
  - name: "营养评估"
    description: "精准计算 BMR、TDEE 和宏量营养素需求"
workflow:
  - step: 1
    action: "评估"
    tool: "calculate_bmr + calculate_tdee"
response_rules:
  - "使用中文，温暖专业"
  - "给出具体的计算数值和科学依据"
```

Agent 代码因此极简化，Diet Agent 仅 20 行，其中 `_build_system_prompt` 只有一行委托调用。

4.4 容错机制

两个独立的容错组件：`CircuitBreaker`（熔断器）在连续 5 次失败后自动熔断 30 秒，防止级联故障；`retry_with_backoff`（装饰器）为每次 LLM 调用提供指数退避重试（等待时间 = 2^{attempt} + 随机抖动）。当所有重试失败后，返回优雅降级回复——如“饮食助手暂时不可用，以下是通用建议：保持均衡饮食、控制盐糖摄入、每天饮水 1.5-2 升”。

五、测试与评估

5.1 测试策略

系统包含 50 项自动化测试，按层次分布：

测试层	数量	覆盖内容	特点
工具函数单元测试	17	BMR/TDEE 计算精度、睡眠评分逻辑、营养分析等	不依赖 LLM API，毫秒级
路由逻辑测试	7	饮食/运动/睡眠/减重/安全等 7 种意图路由	验证 IntentRouter 决策正确性
集成测试	13	记忆管理、RAG 检索、MCP 工具、评估框架	验证模块间协作
模型测试	3	UserProfile BMI 计算、GraphState 初始化	验证 SDD 规格正确性
配置安全测试	2	API Key 零硬编码检查	企业级安全基线
评估基准	8	跨领域综合场景的 Agent 行为评估	话题覆盖 + 路由正确 + 置信度

全部 50 项测试执行时间约 0.8 秒，可在每次代码提交前运行。

5.2 评估基准

10 个标准评估用例覆盖五个维度：

维度	用例	验证内容
纯饮食	diet_001-003	BMR 计算引用、素食增肌方案、食物营养分析
纯运动	ex_001-003	零基础入门、家庭徒手训练、热量消耗计算
纯睡眠	sleep_001-002	失眠 CBT-I 方案、睡眠不足风险警示
多 Agent 协同	multi_001-002	减重（饮食+运动协同）、全面健康改善（三 Agent）
安全场景	safety_001	胸痛关键词触发就医建议、禁止出现"没事""忽略"

评估维度包括话题覆盖率（期望关键词是否出现在回复中）、Agent 路由正确性（是否正确调用了目标 Agent）、置信度达标率和禁止内容检查。综合质量分 = 话题覆盖 × 0.4 + 路由正确 × 0.3 + 置信度 × 0.3，权重分配反映了各维度的重要性。

5.3 评估结果

全部 50 项测试通过。工具函数测试保证了核心计算逻辑（Mifflin-St Jeor 公式、MET 热量估算）的科学准确性。路由测试验证了 7 种典型意图全部正确路由——特别是安全场景（胸痛 → 就医建议）的优先级最高，不会在危急情况下错误地调用专业 Agent。

六、系统升级与扩展

6.1 架构可扩展性

系统在设计层面预留了三个扩展维度。新增 Agent 只需三步：创建类继承 BaseAgent、编写 YAML 提示词、在 Router 注册关键词——不影响现有 Agent。新增工具只需添加实现函数和 Schema 并注册到目标 Agent。切换 LLM Provider 只需修改 `.env` 中的 `LLM_PROVIDER` 变量，所有 Provider 通过 OpenAI 兼容接口统一调用。

6.2 下一阶段计划

优先级	计划	说明
P0	可穿戴设备接入	Apple Health / Google Fit API, 自动获取步数、心率、睡眠数据
P0	饮食拍照识别	多模态 LLM + OCR, 用户拍照自动记录饮食
P1	PWA 移动端适配	手机端便捷使用, 离线缓存支持
P1	多语言国际化	先支持英文和日文
P2	HIPAA 合规改造	医疗数据隐私保护, 支持真实医疗场景
P3	语音交互	ASR + TTS, 更自然的交互方式

6.3 AI 能力演进

阶段 1 (当前) ReAct 工具调用 4 Agent 规则协作 静态知识库	→	阶段 2 (近期) LLM 自主路由 动态 Agent 生成 实时知识更新	→	阶段 3 (远期) 自主学习 联邦学习 预测性健康预警
--	---	--	---	--------------------------------------

七、课程总结

7.1 个人收获

本课程让我完成了从"代码编写者"到"智能体编排者"的思维跃迁。在传统开发中我们写确定性逻辑（输入 A → 计算 → 输出 B），而在 Agentic AI 开发中，我们设计的是**非确定性系统的协作框架**——你不知道 LLM 具体会回复什么，但你要确保它调用了正确的工具、产出有置信度标注、多 Agent 输出经过了质量审查。

SDD 方法论的内化是另一重要收获。项目中所有数据模型先定义为 Pydantic 类再在代码中使用——它们既是设计文档也是运行时校验。当 `AgentMessage(**data)` 自动检查字段类型和约束时，"规格即代码"不再是抽象概念。

工程实践方面，熔断器、指数退避重试、优雅降级、提示词配置化、可观测性追踪——这些企业级模式在项目中从理论变成了真正运行并发挥作用的机制。

7.2 工程思维转变

之前	之后
"代码跑通就行"	"API 失败了怎么办？熔断后如何恢复？"
"提示词写在代码里"	"提示词是配置，应该独立管理、可热更新"
"手动测一下"	"50 个自动化测试 + 10 个评估基准，每次改动自动验证"
"先写代码再补文档"	"Pydantic 模型就是文档，SDD 驱动开发"

7.3 对课程的建议

1. **允许 2 人组队**——真实的企业级 Agent 系统很难单人完成，团队协作本身也是重要的工程能力。如果担心评分公平性，可以要求每组提交分工说明和 Git commit 记录以追踪个人贡献，同时让每位组员在报告中独立撰写"个人工作与收获"章节。
2. **增加 LangGraph 框架前置 tutorial**——对于没有接触过有状态图编排框架的同学，LangGraph 的 StateGraph、条件边、checkpoint 等概念需要一定消化时间。建议在第 3-4 周安排一次 2 课时的 hands-on workshop，从"单 Agent + 工具调用"逐步演进到"Supervisor-Worker 多 Agent 架构"，降低期末大作业的启动门槛。
3. **Agent 评估专题**——如何科学评估 Agent 的"好坏"是一个开放问题，建议增加一次专题讨论，涵盖自动评估（关键词覆盖、工具调用正确率、置信度校验）和人工评估（回复质量、安全性、实用性）的互补关系。如果能引入学术界常用的 AgentBench 或 SWE-bench 等基准做一次课堂演示，学生会更深刻理解评估的复杂度。
4. **提示词工程专项讲解**——Agent 系统的行为很大程度上由提示词决定，但课程中提示词设计更多依赖学生自己摸索。建议增加一次提示词工程专题（如 few-shot 策略、chain-of-thought 触发的写法、如何避免 Agent 幻觉和多 Agent 提示词的隔离设计），配合课堂即时实验，学生的 Agent 质量会有明显提升。